

Neutral Prediction Deduplication Review

Prepared: July 14, 2026 | Scope: Should neutral predictions reserve dedup slots?

1. Current Behavior

1.1 Dedup Logic — No Type Awareness

The dedup check in `PredictionGenerator.GeneratePredictionsForWatchlistAsync()` (lines 420–474) operates on exactly two dimensions: ticker and time_window. It never inspects prediction_type. The code:

```
var allExisting = recentPredictions
    .Concat(openPredictions)
    .DistinctBy(p => p.Id)
    .ToList();

var existingTimeWindowsByTicker = allExisting
    .GroupBy(p => p.Ticker.ToUpperInvariant())
    .ToDictionary(
        g => g.Key,
        g => new HashSet<string>(g.Select(p => p.TimeWindow)));
```

The lookup stores only time_window strings per ticker. When a new prediction is generated, the check is:

```
if (existingTimeWindowsByTicker.TryGetValue(tickerKey, out var existingWindows)
    && existingWindows.Contains(pred.TimeWindow))
    continue; // skip — slot occupied
```

There is no filter on PredictionType. A neutral_high_volatility prediction for AAPL/1_week reserves the exact same slot that a bullish prediction for AAPL/1_week would. Once that slot is occupied, no other prediction — regardless of type or direction — can be issued for that ticker+time_window until the original prediction is either evaluated or expired.

1.2 What Occupies the Dedup Set

The dedup set is built from two sources: (1) all predictions created today (any status, including evaluated/expired), and (2) all predictions with status="open" (any date). A neutral prediction contributes to both sources. Today's neutrals block via source (1), and older open neutrals block via source (2).

1.3 How Neutral Types Are Assigned

The `ScoringEngine.DetermineDirectionAndType()` method (lines 506–544) assigns neutral types as a fallback when directional thresholds are not met:

- If both bullish and bearish scores are below 8 → watch_only
- If Bollinger Bandwidth > 8 (wide bands) → neutral_high_volatility
- If trend signals exist but directional margin < 5 → neutral_range_bound
- Default fallback → neutral_no_edge

Critically, the `ScoringEngine` still computes full bullish and bearish scores for neutral predictions. These scores are stored in `BullishScore` and `BearishScore` on the `PredictionCandidate`. The engine chose "neutral" because neither score was strong enough, not because no directional signal existed.

1.4 Neutral Evaluation Timeline

Neutral predictions follow the same maturity windows as directional ones (e.g., 1_week = 120h minimum). They are evaluated by the NeutralOutcomeEvaluator in EOD Step 6, after the directional OutcomeEvaluator runs in Step 4. Until evaluated, they remain open and block the dedup slot.

Current data: 13 of 33 open predictions are neutral_high_volatility. These occupy dedup slots for their ticker+1_week combination for at least 5 full trading days.

2. Architectural Intent

This was not an intentional design decision.

The evidence:

- The dedup code has no comments mentioning neutral predictions. The only comment says "This prevents duplicates within the same day even if prior predictions were already evaluated/closed." The word "neutral" does not appear in the dedup block.
- The dedup logic was written to prevent the engine from issuing two competing directional predictions (e.g., bullish 1_week + bearish 1_week for the same ticker). Preventing two bullish 1_week predictions for AAPL makes sense. Preventing a bullish 1_week after a neutral 1_week is a side effect, not an intent.
- The NeutralOutcomeEvaluator was added later as a "parallel pipeline" (comment on DynamicPickOrchestrator line 237: "parallel pipeline — does not touch directional evaluator"). It was designed to evaluate neutral predictions independently. But no corresponding change was made to the dedup logic to account for the fact that neutrals and directionals serve fundamentally different purposes.
- The ScoringEngine assigns neutral types as fallbacks ("we don't have enough signal to call a direction"). Treating a fallback classification as a first-class dedup reservation was not a deliberate architectural choice.

Conclusion: Neutral predictions inherited dedup behavior designed for directional predictions. The dedup system was not updated when the neutral classification and evaluation pipelines were introduced.

3. Learning Impact

3.1 Signal Observation Extraction — Neutrals Are Invisible

The `LearningEngine.ExtractSignalObservationsAsync()` reads from `GetRecentPredictionsAsync()` and `GetRecentOutcomesAsync()`. The outcomes table (`prediction_outcomes`) only contains directional predictions — neutral outcomes go to a separate `neutral_prediction_outcomes` table. The `LearningEngine` never reads `neutral_prediction_outcomes`.

This means: if the engine issues a neutral prediction on Monday, then a bullish signal emerges on Tuesday but is blocked by dedup, the `LearningEngine` never sees a directional outcome for that ticker+window. The signal observation that would have been "bullish signal + correct direction" is never recorded.

3.2 Adaptive Learning — Also Blind to Neutrals

The `AdaptiveLearningEngine` records outcomes only for directional predictions (it reads `PredictionOutcome.Outcome` which is "win"/"loss"). Neutral predictions are evaluated via `NeutralOutcomeEvaluator` into `neutral_prediction_outcomes` with a `NeutralAccuracyScore`, `CounterfactualDirection`, and `OpportunityMissedScore`. None of this data feeds into the `AdaptiveLearningEngine`'s conditional performance tracking.

Result: the learning system cannot learn conditional patterns like "when we call `neutral_high_volatility` in this regime, we miss opportunities X% of the time."

3.3 Opportunity Learning — Subtle Misclassification

The `OpportunityLearningService` (line 236) checks whether a prediction was directionally correct:

```
var predBullish = bestPrediction.PredictionType == PredictionType.bullish;
var moveBullish = direction == "up";
predictionCorrectDirection = predBullish == moveBullish;
```

For a `neutral_high_volatility` prediction: `PredictionType != bullish`, so `predBullish = false`. If the stock moved up, `predictionCorrectDirection = false`, and the opportunity is classified as "WrongDirection." If the stock moved down, `predictionCorrectDirection = true`, and it's classified as "Captured." Neither classification is accurate — the prediction didn't express a direction at all. The opportunity learning data is subtly corrupted for any ticker with a neutral prediction.

3.4 The "Neutral Yesterday, Bullish Today" Scenario

Consider this sequence:

- Monday morning scan: AAPL scored bull=15, bear=12, margin=3 (below `MinEdgeMargin` of 15). `Bollinger Bandwidth > 8`. Engine issues `neutral_high_volatility / 1_week`.
- Tuesday: AAPL reports strong earnings after-hours. Wednesday morning: catalyst evaluators would now produce bull=45, bear=10, margin=35. This is a strong bullish signal.
- Wednesday morning scan: `PredictionGenerator` generates a bullish/1_week prediction for AAPL. But dedup finds the Monday neutral/1_week is still open (only 48h old, needs 120h). The bullish prediction is silently discarded.
- The bullish signal is never recorded. The `LearningEngine` never sees that catalyst+momentum → bullish was correct. The system's view of AAPL remains "neutral" until Friday at earliest.

The information loss is permanent: not just delayed, but never captured at all for that `time_window`.

4. Market Responsiveness Impact

4.1 Earnings

Earnings are the single largest catalyst for directional repricing. A pre-earnings neutral call ("we don't know which way it'll go") is reasonable. But post-earnings, the direction is typically clear within hours. The current architecture cannot issue a directional prediction for 5+ days after a neutral call, which means the system misses the entire post-earnings move for the primary time_window.

Impact: High. Earnings-driven moves are exactly the kind of signal the system is designed to capture. A neutral_high_volatility call pre-earnings that blocks a post-earnings bullish call is a direct contradiction of the system's purpose.

4.2 News and Catalysts

Breaking news (FDA approvals, contract wins, management changes) can shift a ticker from neutral to strongly directional overnight. The system has the data infrastructure to detect this — the MarketSnapshotBuilder fetches news from Finnhub and StockFit, the CatalystEvaluator scores catalyst strength, and the ScoringEngine weights these signals. But all of this intelligence is wasted if a neutral prediction from the day before is blocking the slot.

Impact: High. The system invests significant compute in building real-time market snapshots and scoring catalysts, then throws away the result because of a stale neutral claim.

4.3 Sector Rotation

Sector rotation tends to develop over days. A ticker might be neutral at the start of a rotation (no clear signal yet) but become strongly directional as the rotation confirms. Sector context is already modeled (MarketContextEvaluator uses SPY/QQQ benchmarks). The 1_week dedup window (120h) spans a full trading week — enough time for a sector rotation to fully develop and become undeniable, yet the engine cannot act on it for that ticker.

Impact: Moderate. Sector rotation is slower than earnings/news, so the 1_week window may overlap with the move developing. But for 1_month windows (504h / 21 days), the impact is significant — the system could miss the entire rotation.

4.4 Macro Events

Fed decisions, CPI reports, and geopolitical events affect all tickers simultaneously. A neutral call on Monday may be invalidated by a Wednesday FOMC announcement. The engine's MarketContextEvaluator would reflect the changed macro environment in Wednesday's scoring, but dedup prevents acting on it.

Impact: Moderate to High. Macro events can shift entire market regimes. The learning system already tracks regime-conditional performance (AdaptiveLearningEngine records MarketRegime conditions), but if neutrals block directional predictions, the system can't respond to regime changes.

4.5 Technical Reversals

Technical reversals (breakdowns below support, breakouts above resistance) are precisely what moves a ticker from "range-bound" or "no edge" to directional. The ScoringEngine already scores trend, momentum, and volatility signals for every ticker. A neutral_range_bound call followed by a breakout the next day is a textbook case where the engine should upgrade to a directional call — but cannot.

Impact: High for short-term windows (3_day), moderate for 1_week. Technical reversals tend to play out quickly.

4.6 Quantified Impact

Current data: 13 of 33 open predictions are neutral_high_volatility, all with 1_week windows. These 13 predictions block the system from issuing any 1_week directional prediction for 13 tickers for 5+ trading days. If even 2–3 of these tickers experience a material catalyst during that window, the system misses the directional signal entirely.

Over time, with ~5 new neutral 1_week predictions per day, the system could have 25+ tickers continuously blocked from receiving directional 1_week predictions. For a universe of ~15 active tickers, this means most tickers are blocked most of the time.

5. Alternative Approaches

Option A: Status Quo — Neutrals Reserve Slots Identically

Advantages	Disadvantages
No code changes required. No risk of regression.	Blocks directional predictions after catalysts. Learning pipeline loses data permanently.
Prevents conflicting predictions for same ticker+window (prediction coherence).	Neutral predictions are not actionable — "coherence" with a non-actionable prediction has no user value.
Simpler mental model for the dedup system.	OpportunityLearningService misclassifies neutrals as directional (subtle data quality issue).

Assessment: This option preserves prediction coherence at the cost of market responsiveness and learning quality. The coherence benefit is weak because neutral predictions express absence of conviction, not a competing thesis.

Option B: Neutrals Never Reserve Dedup Slots

Advantages	Disadvantages
Engine can immediately issue a directional prediction when signals strengthen. Maximum market responsiveness.	Could produce a neutral + directional prediction for the same ticker+window on the same day if conditions change intra-day.
Learning pipeline captures directional signals that would otherwise be lost.	Two coexisting predictions (neutral + bullish) for the same ticker+window may confuse downstream consumers (paper stock candidates, portfolio lifecycle).
Simple to implement: filter allExisting to exclude neutral types before building the dedup set.	If neutral predictions remain open, GetOpenPredictionsAsync() would return both the neutral and the directional prediction. The evaluator processes all open predictions, so both would eventually be evaluated — potentially producing conflicting outcomes.
Aligns dedup behavior with the actual purpose of neutral predictions ("we don't know" is not a position that should block future positions).	Requires deciding what happens to the original neutral when a directional prediction supersedes it: leave it open (dual evaluation), close it as "superseded", or expire it.

Assessment: This option maximizes responsiveness but introduces a supersession problem. The neutral prediction still exists and still has an open status. Without a mechanism to close or supersede the original neutral when a directional prediction is issued, the system would have two active predictions for the same slot.

Option C: Neutrals Reserve Slots Until Thesis Materially Changes

Advantages	Disadvantages
Only issues a new directional prediction when a real catalyst or signal change justifies it, avoiding noise.	Requires defining "material change" — a new threshold or scoring delta that determines when the original neutral thesis is invalidated.
Maintains prediction coherence for truly neutral situations while allowing upgrades when warranted.	More complex to implement: the dedup check must score the new prediction, compare to the original neutral's scores, and decide whether the delta exceeds a threshold.
Produces a clear audit trail: "neutral superseded by bullish because catalyst strength increased by X."	The threshold is a new hyperparameter that needs tuning. Too low → behaves like Option B. Too high → behaves like Option A.

Could automatically close/expire the original neutral with a "superseded" status, solving the dual-prediction problem.	Requires the morning scan to look up the original neutral's scores and compare — adds a query and comparison per dedup-blocked prediction.
--	--

Assessment: This is the most nuanced option. It respects the original neutral classification while allowing directional upgrades when the market provides new information. The main cost is implementation complexity and the need to tune a new threshold.

6. Recommendation

Neutral predictions should NOT reserve dedup slots the same way directional predictions do.

The current behavior is an accidental inheritance, not a design choice. It costs the system market responsiveness (Section 4), learning data quality (Section 3), and opportunity capture accuracy (Section 3.3) — all in exchange for "coherence" with a prediction that explicitly expresses no directional conviction.

The recommended approach is a modified Option B + supersession:

- Neutral predictions do not occupy dedup slots. The dedup set should filter out neutral types (`watch_only`, `neutral_no_edge`, `neutral_high_volatility`, `neutral_range_bound`, `neutral`) before building `existingTimeWindowsByTicker`.
- When a directional prediction is issued for a ticker+window that has an open neutral prediction, the neutral prediction is automatically set to "expired" with a reason like `"superseded_by_directional"`. This avoids dual-prediction confusion.
- Neutral-to-neutral dedup should remain: a `neutral_high_volatility` should still block another `neutral_high_volatility` for the same ticker+window, since issuing the same non-actionable classification twice has no value.
- The `NeutralOutcomeEvaluator` should record the supersession as data: "this neutral was replaced after N hours by a bullish prediction" — valuable for learning whether the engine's neutrality threshold is too conservative.

This approach gives the engine full responsiveness to catalysts while maintaining clean prediction semantics (one active prediction per type per slot, no conflicting pairs).

6.1 Bugs to Address Regardless of Approach

The `OpportunityLearningService` (line 236) treats neutral predictions as implicitly bearish by checking `PredictionType == bullish`. If `predBullish` is false (which it is for all neutral types), the service classifies a stock-up move as `"WrongDirection"` and a stock-down move as `"Captured"`. This is incorrect. Neutral predictions should either be excluded from the capture-status logic or evaluated separately. This bug exists independent of the dedup question and should be fixed in any case.